

## CANDIDATE SUBMISSION

# Agnos Backend Deliverables

Hospital middleware implemented in Go, Gin, PostgreSQL, Nginx, and Docker Compose.

|            |                                  |
|------------|----------------------------------|
| Candidate  | PS-Safe                          |
| Contact    | phatdanai.shin@gmail.com         |
| Repository | PS-safe/Agnos-Backend-Assignment |
| Prepared   | 15 August 2026                   |

## DELIVERABLE INDEX

### Everything required for reviewer handoff

|    |                                                                                      |
|----|--------------------------------------------------------------------------------------|
| 01 | <b>Project structure</b><br>Package boundaries, responsibilities, and request flow   |
| 02 | <b>API specification</b><br>Routes, validation, authorization, responses, and errors |
| 03 | <b>ER diagram</b><br>Hospital-scoped staff and patient data model                    |
| 04 | <b>Docker Compose</b><br>Nginx, Go API, PostgreSQL, startup, and verification        |
| 05 | <b>GitHub</b><br>Private repository, commit history, CI, and reviewer access         |

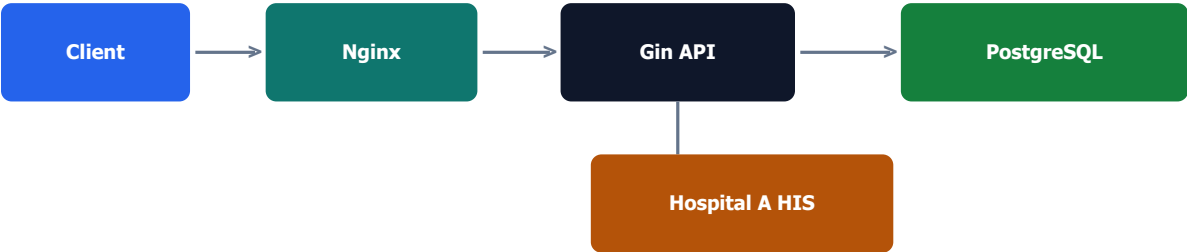
#### Implementation status

All Go tests, race tests, static analysis, container build, Compose validation, and the isolated runtime smoke test pass in GitHub Actions.

DELIVERABLE 1

# Project structure

A layered Go service with explicit application boundaries and infrastructure adapters.



Request flow: routing -> validation/auth -> business rules -> storage or hospital adapter

|                                                                                                                                                                                                                                                                                                                                                        | Path                      | Responsibility                                                                |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------|-------------------------------------------------------------------------------|
| <pre>.  -- cmd/api/      -- main.go  -- internal/      -- config/      -- core/      -- hospital/      -- httpapi/      -- patient/      -- security/      -- staff/      -- storage/postgres/  -- migrations/  -- deploy/nginx.conf  -- scripts/smoke-test.sh  -- docs/  -- Dockerfile  -- docker-compose.yml  -- go.mod / go.sum `-- README.md</pre> | cmd/api                   | Composition root, dependency wiring, HTTP server lifecycle, graceful shutdown |
|                                                                                                                                                                                                                                                                                                                                                        | internal/httpapi          | Gin routes, request validation, JWT middleware, DTO mapping, safe logging     |
|                                                                                                                                                                                                                                                                                                                                                        | internal/staff            | Staff registration and login business rules                                   |
|                                                                                                                                                                                                                                                                                                                                                        | internal/patient          | Hospital-scoped search orchestration and upstream refresh                     |
|                                                                                                                                                                                                                                                                                                                                                        | internal/hospital         | Hospital A HTTP adapter and response normalization                            |
|                                                                                                                                                                                                                                                                                                                                                        | internal/security         | bcrypt password hashing and signed JWT issue/verification                     |
|                                                                                                                                                                                                                                                                                                                                                        | internal/storage/postgres | Staff and patient persistence using pgx                                       |
|                                                                                                                                                                                                                                                                                                                                                        | migrations                | Database schema, indexes, constraints, and Hospital A seed                    |
|                                                                                                                                                                                                                                                                                                                                                        |                           |                                                                               |

**Boundary rule**

HTTP handlers depend on application interfaces; application services depend on repository and hospital-client interfaces. Infrastructure implementations remain replaceable and unit-testable.

DELIVERABLE 2

# API specification

JSON over HTTP through Nginx at http://localhost:8080.

| Method | Route           | Access     | Success | Purpose                                   |
|--------|-----------------|------------|---------|-------------------------------------------|
| GET    | /health         | Public     | 200     | Process liveness                          |
| POST   | /staff/create   | Public*    | 201     | Create hospital staff credentials         |
| POST   | /staff/login    | Public     | 200     | Issue hospital-scoped access token        |
| GET    | /patient/search | Bearer JWT | 200     | Search only within authenticated hospital |

| POST                                                                                                         | /staff/create                                                                                                                                                                     | Public assignment route |
|--------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------|
| <pre>{   "username": "doctor.one",   "password": "correct-horse-battery",   "hospital": "hospital-a" }</pre> | <pre>HTTP/1.1 201 Created {   "data": {     "id": "&lt;uuid&gt;",     "username": "doctor.one",     "hospital": "hospital-a",     "created_at": "&lt;RFC3339 UTC&gt;"   } }</pre> |                         |

| Field    | Rules                                                                                   |
|----------|-----------------------------------------------------------------------------------------|
| username | Required; lowercase; 3-64 letters, numbers, dot, underscore, or hyphen; globally unique |
| password | Required; valid UTF-8; 12-72 bytes; bcrypt cost 12; never returned                      |
| hospital | Required existing hospital code; normalized to lowercase                                |

| POST /staff/login                                                                                                                                                                 |                                                                                                                                                   | Public |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|--------|
| <b>Request</b><br>{ "username": "doctor.one",<br>"password": "correct-horse-battery",<br>"hospital": "hospital-a" }                                                               | <b>Success 200</b><br>Returns access_token, token_type=Bearer, expires_in, and the sanitized staff record.                                        |        |
| <b>Authentication behavior</b><br>Username and hospital are normalized. Password comparison uses bcrypt. Invalid username, password, or hospital shares one generic 401 response. | <b>Token scope</b><br>HS256 JWT includes staff ID, hospital ID, hospital code, username, issuer, and expiration. Hospital scope is authoritative. |        |

Production note

The assignment requires staff creation but defines no administrator contract. The route is public for assignment compatibility and should be restricted by an administrator or invitation workflow in production.

## DELIVERABLE 2

# Patient search contract

Hospital scope is derived only from the verified access token.

GET

`/patient/search`

Authorization: Bearer &lt;token&gt;

**Search invariant**

Every filter is individually optional, but at least one filter is required. Unknown parameters are rejected. Results are capped at 100.

| Query parameter            | Match behavior                                     | Hospital A behavior                                     |
|----------------------------|----------------------------------------------------|---------------------------------------------------------|
| <code>national_id</code>   | Exact                                              | Preferred identifier; triggers upstream lookup          |
| <code>passport_id</code>   | Exact                                              | Triggers lookup when <code>national_id</code> is absent |
| <code>first_name</code>    | Case-insensitive substring across Thai and English | Database cache only                                     |
| <code>middle_name</code>   | Case-insensitive substring across Thai and English | Database cache only                                     |
| <code>last_name</code>     | Case-insensitive substring across Thai and English | Database cache only                                     |
| <code>date_of_birth</code> | Exact YYYY-MM-DD                                   | Database cache only                                     |
| <code>phone_number</code>  | Exact source value                                 | Database cache only                                     |
| <code>email</code>         | Case-insensitive exact                             | Database cache only                                     |

**Response fields**

| Group                 | Fields                                                                                                                                                                      |
|-----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Names                 | <code>first_name_th</code> , <code>middle_name_th</code> , <code>last_name_th</code> , <code>first_name_en</code> , <code>middle_name_en</code> , <code>last_name_en</code> |
| Identity              | <code>date_of_birth</code> , <code>patient_hn</code> , <code>national_id</code> , <code>passport_id</code>                                                                  |
| Contact / demographic | <code>phone_number</code> , <code>email</code> , <code>gender</code>                                                                                                        |
| Envelope              | <code>data.patients[]</code> and <code>data.count</code> ; no match returns an empty array with count 0                                                                     |

**Hospital A upstream contract**

GET {HOSPITAL\_A\_BASE\_URL}/patient/search/{national\_id\_or\_passport\_id}

Accept: application/json

X-API-Key: <optional HOSPITAL\_A\_API\_KEY>

Successful payload -> validate -> normalize -> assign hospital\_id -> upsert -> filtered query

| HTTP | Code                                                  | Meaning                                                             |
|------|-------------------------------------------------------|---------------------------------------------------------------------|
| 400  | <code>invalid_request</code>                          | Malformed JSON/date, empty search, unknown field or query parameter |
| 401  | <code>unauthorized</code>                             | Missing, malformed, expired, or invalid credentials/token           |
| 404  | <code>not_found</code> / <code>route_not_found</code> | Unknown hospital during creation or unknown route                   |
| 409  | <code>conflict</code>                                 | Staff username already exists                                       |
| 429  | <code>Nginx response</code>                           | Per-client route rate limit exceeded                                |
| 502  | <code>hospital_unavailable</code>                     | Hospital HIS request or payload failed validation                   |
| 500  | <code>internal_error</code>                           | Unexpected application or persistence failure                       |

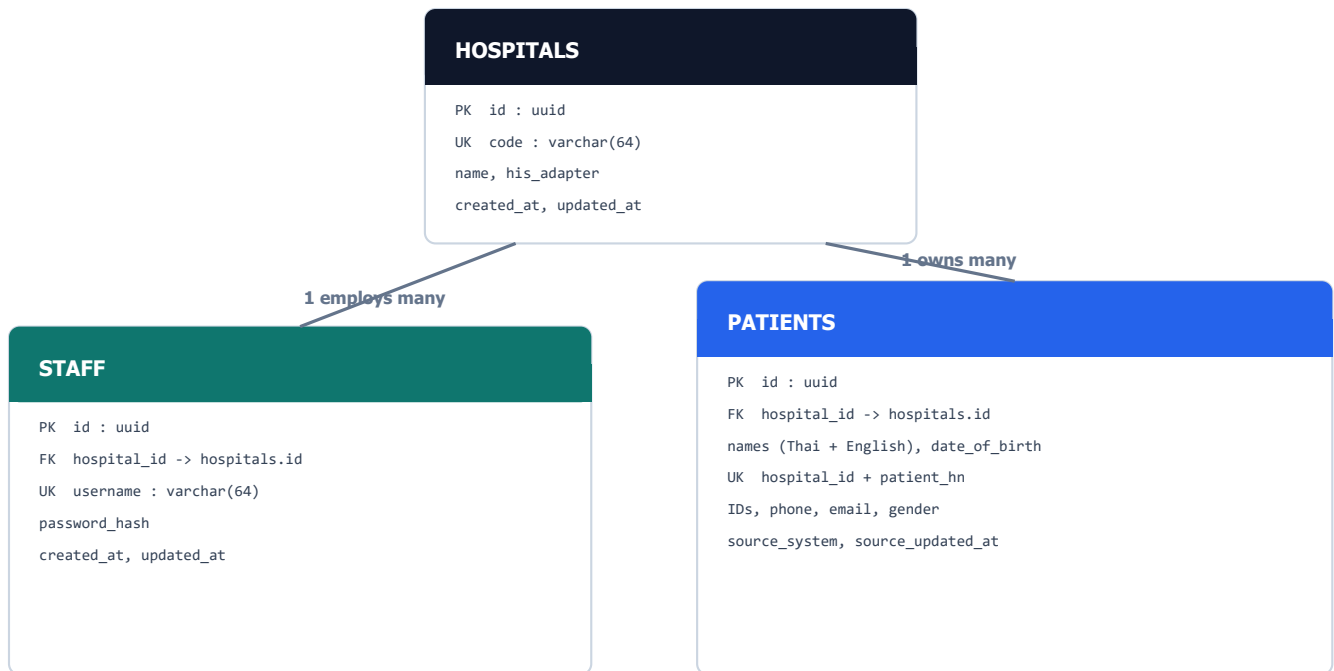
**Common transport behavior**

JSON only; one-megabyte request limit; RFC 3339 timestamps; Cache-Control: no-store; request IDs on responses; logs record route templates without query strings or credentials.

## DELIVERABLE 3

# Entity-relationship diagram

The relational model enforces hospital ownership at every staff and patient boundary.



| Constraint                             | Purpose                                                                                 |
|----------------------------------------|-----------------------------------------------------------------------------------------|
| staff.username is globally unique      | Login also requires the account's matching hospital code.                               |
| staff.hospital_id -> hospitals.id      | Every staff member belongs to one hospital                                              |
| patients.hospital_id -> hospitals.id   | Every patient row is owned by one hospital tenant                                       |
| unique (hospital_id, patient_hn)       | Stable hospital-local patient identity and safe upsert target                           |
| unique hospital + national/passport ID | Prevents duplicate identifiers within the same hospital while allowing null identifiers |
| national_id or passport_id required    | Rejects patient rows without any supported external identifier                          |
| gender in M or F                       | Matches the stated Hospital A response contract                                         |

## Tenant-safe data flow

|   |                |                                                                                          |
|---|----------------|------------------------------------------------------------------------------------------|
| 1 | Authenticate   | Verify JWT signature, issuer, expiration, staff ID, hospital ID, and hospital code       |
| 2 | Resolve source | For an identifier search, select the adapter mapped to the token's hospital code         |
| 3 | Normalize      | Validate Hospital A payload and assign the authenticated hospital ID                     |
| 4 | Persist        | Upsert by hospital ID and hospital number; never accept hospital scope from client input |
| 5 | Query          | Apply hospital ID plus every supplied filter and cap results at 100                      |

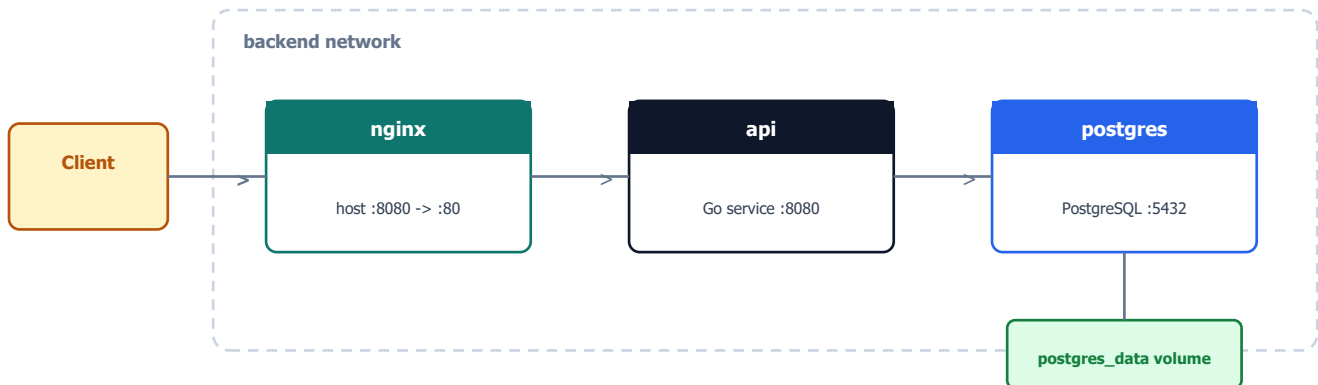
### Primary isolation guarantee

Patient repository queries always include hospital\_id from the verified token. Runtime verification confirmed that Hospital A and a synthetic Hospital B cannot see each other's matching patient records.

## DELIVERABLE 4

# Docker Compose setup

A three-service stack with ordered health checks, one private network, and persistent database storage.



| Service  | Image / build          | Responsibilities and dependencies                                                                                     |
|----------|------------------------|-----------------------------------------------------------------------------------------------------------------------|
| nginx    | nginx:1.27-alpine      | Publishes host port 8080; routing, request IDs, body limit, timeouts, and rate limits; waits for healthy API          |
| api      | Multi-stage Dockerfile | Builds Go 1.24 static binary; runs as non-root user app; connects to PostgreSQL and Hospital A; exposes internal 8080 |
| postgres | postgres:16-alpine     | Initializes schema from migration; health checked with pg_isready; stores data in postgres_data                       |

## Required startup

```
copy .env.example .env          # Windows PowerShell: Copy-Item .env.example .env
# Replace POSTGRES_PASSWORD and JWT_SECRET (JWT secret: at least 32 characters)
docker compose up --build
curl http://localhost:8080/health
```

| Variable                | Required | Purpose / default                           |
|-------------------------|----------|---------------------------------------------|
| POSTGRES_PASSWORD       | Yes      | Database password; no committed default     |
| JWT_SECRET              | Yes      | HS256 signing secret; minimum 32 characters |
| APP_PORT                | No       | Published Nginx port; default 8080          |
| JWT_ISSUER / JWT_EXPIRY | No       | agnos-hospital-middleware / 8h              |
| HOSPITAL_A_BASE_URL     | No       | https://hospital-a.api.co.th                |
| HOSPITAL_A_API_KEY      | No       | Optional X-API-Key value; never commit it   |
| HOSPITAL_HTTP_TIMEOUT   | No       | Hospital client timeout; default 5s         |

### Startup ordering

PostgreSQL must become healthy before the API starts. The API must become healthy before Nginx starts. Restart policies are set to unless-stopped.

## DELIVERABLE 4

# Compose operations and verification

A repeatable smoke test verifies the deployed path through Nginx and removes its own test resources.

## Reviewer smoke test

```
./scripts/smoke-test.sh
```

```
# Optional when port 18080 is already in use:  
SMOKE_APP_PORT=18081 ./scripts/smoke-test.sh
```

| Smoke-test phase       | What is verified                                                                     |
|------------------------|--------------------------------------------------------------------------------------|
| Isolated environment   | Unique Compose project, disposable PostgreSQL volume, host port 18080 by default     |
| Health and proxy       | Nginx -> API -> health response, X-Request-ID, and Cache-Control: no-store           |
| Negative authorization | Patient search without a Bearer token returns 401                                    |
| Staff flow             | Staff creation succeeds; login returns a non-empty JWT                               |
| Input safety           | Authenticated empty search returns 400                                               |
| Persistence            | Synthetic patient is inserted into PostgreSQL and returned through the protected API |
| Cleanup                | Containers, network, and test volume are removed even when the script fails          |

## Observed runtime evidence

| Check                           | Result                                                                      |
|---------------------------------|-----------------------------------------------------------------------------|
| Docker Compose build and health | PASS - PostgreSQL and API healthy; Nginx served on the configured host port |
| Staff create/login and JWT      | PASS - positive and negative cases exercised through Nginx                  |
| Hospital isolation              | PASS - Hospital A and synthetic Hospital B returned only their own patient  |
| Hospital A adapter              | PASS - mocked HTTP lookup normalized and upserted the patient record        |
| Restart persistence             | PASS - patient remained available after API container restart               |
| Runtime privacy                 | PASS - no tested credential, token, or identifier appeared in service logs  |
| Container user                  | PASS - API process runs as non-root user app                                |

## Common operations

```
docker compose ps          # service status  
docker compose logs -f api nginx # runtime logs  
docker compose restart api  # restart application only  
docker compose down        # stop stack, keep database volume  
docker compose down -v     # stop and remove database volume (destructive)
```

### Data safety

Use docker compose down without -v when data must be preserved. The -v option permanently removes the named PostgreSQL volume.

## DELIVERABLE 5

# GitHub submission

A private main branch with chronological feature commits and automated verification.

|                        |                                                                                                                       |
|------------------------|-----------------------------------------------------------------------------------------------------------------------|
| Repository             | <a href="https://github.com/PS-safe/Agnos-Backend-Assignment">https://github.com/PS-safe/Agnos-Backend-Assignment</a> |
| Visibility             | PRIVATE                                                                                                               |
| Default branch         | main                                                                                                                  |
| Candidate identity     | PS-Safe <phatdanai.shin@gmail.com>                                                                                    |
| Latest verified commit | 7d36781 - fix: require hospital during staff login                                                                    |
| Latest CI run          | <a href="#">Successful workflow run 31896139082</a>                                                                   |

## Commit progression

| Stage       | Representative commits                                                                |
|-------------|---------------------------------------------------------------------------------------|
| Foundation  | initialize Go project; architecture and API decisions; configuration and domain model |
| Data        | PostgreSQL schema and hospital seed; PostgreSQL repositories                          |
| Security    | staff account service; JWT authentication                                             |
| Integration | Hospital A client; hospital-scoped patient search                                     |
| Transport   | staff and patient HTTP APIs                                                           |
| Operations  | Docker Compose and Nginx; CI workflow; reproducible Compose smoke test                |
| Handoff     | reviewer documents; GitHub history integration; Node.js 24 action maintenance         |

## CI pipeline

| Step | Automated gate                                           |
|------|----------------------------------------------------------|
| 1    | Verify module checksums                                  |
| 2    | Run all Go tests with race detection and coverage output |
| 3    | Run go vet static analysis                               |
| 4    | Build the Go application                                 |
| 5    | Validate Docker Compose configuration                    |
| 6    | Build the production container image                     |
| 7    | Run isolated Docker Compose smoke test                   |

### Reviewer access

Because the assignment is confidential, keep the repository private and add only the intended interviewer as a collaborator before submission.

## DELIVERABLE

# Submission checklist

One-page map from the requested deliverables to their repository evidence.

| # | Required deliverable | Evidence in this PDF                                       | Repository source                                 |
|---|----------------------|------------------------------------------------------------|---------------------------------------------------|
| 1 | Project structure    | Layer diagram, file tree, package responsibilities         | README.md; cmd/; internal/                        |
| 2 | API specification    | Routes, schemas, filters, authorization, error model       | docs/API_SPEC.md; docs/openapi.yaml               |
| 3 | ER diagram           | Hospitals, staff, patients, keys, constraints, tenant flow | docs/ER_DIAGRAM.md; migrations/                   |
| 4 | Docker Compose setup | Topology, variables, startup, operations, smoke test       | docker-compose.yml; Dockerfile; deploy/; scripts/ |
| 5 | GitHub               | Private repository, main branch, commits, CI evidence      | .github/workflows/ci.yml                          |

## Reviewer quick start

```
git clone https://github.com/PS-safe/Agnos-Backend-Assignment.git
cd Agnos-Backend-Assignment
cp .env.example .env
# Set strong POSTGRES_PASSWORD and JWT_SECRET values in .env
docker compose up --build
curl http://localhost:8080/health
```

## Verification summary

| Verification                                 | Result                 |
|----------------------------------------------|------------------------|
| Go unit tests                                | PASS                   |
| Aggregate statement coverage                 | 76.7%                  |
| Linux race detection                         | PASS in GitHub Actions |
| go vet                                       | PASS                   |
| Application and container builds             | PASS                   |
| Compose configuration and runtime smoke test | PASS                   |
| Latest GitHub Actions annotations            | None                   |

### Submission status

All five requested deliverables are implemented, documented, committed, pushed, and independently verified by the repository CI workflow.

## End of deliverables

Agnos Hospital Middleware | Backend Developer Assignment